Vue.js Frontend Sablon

0. Mire való ez a sablon

Ez a dokumentum Vue.js alapú frontend projektek felépítéséhez ad bemásolható parancsokat és kódrészleteket. A lépések célja: projekt létrehozása, komponensek használata, adatkötés, eseménykezelés, API hívás, űrlapkezelés és egyszerű dizájn.

1. Szükséges programok

• Node.js: a Vue fejlesztői szerver és az npm csomagkezelő miatt kell.

• VS Code: projekt megnyitásához és beépített terminál használatához.

• Git opcionális, de hasznos verziókezeléshez.

• Composer csak akkor kell a frontend mellé, ha Laravel projektben dolgozol. Tiszta Vue projektnél npm kell, nem Composer.

Ellenőrzés terminálban

```
node -v
npm -v
```

Várt eredmény: verziószámok jelennek meg, például v20.x.x és 10.x.x. Ha hiba jön, a Node.js nincs telepítve vagy nincs PATH-ba rakva.

2. Új Vue projekt létrehozása

VS Code terminálban vagy sima terminálban menj abba a mappába, ahová a projektet szeretnéd.

```
cd Desktop
npm create vue@latest frontend-projekt
cd frontend-projekt
npm install
npm run dev
```

Várt eredmény: a terminál kiír egy helyi címet, például http://localhost:5173. Ezt böngészőben megnyitva látszik az oldal.

Gyakori hiba: npm is not recognized. Megoldás: Node.js újratelepítés, majd terminál újranyitása.

3. Alap mappaszerkezet

```
frontend-projekt/
  node_modules/          // telepített npm csomagok, nem kell szerkeszteni
  public/                // statikus fájlok
  src/
    assets/              // képek, css, ikonok
    components/          // újrahasznosítható Vue komponensek
    views/               // oldalak, ha routert használsz
    router/              // útvonalak
    App.vue              // fő komponens
    main.js              // belépési pont
  package.json           // függőségek és script parancsok
  vite.config.js         // Vite beállítás
```

4. App.vue alap sablon

Az App.vue a fő komponens. Ide kerülhet a layout, navbar, fő tartalom vagy router-view.

```
<script setup>
const appNev = 'Frontend gyakorló projekt'
</script>

<template>
  <main class="page">
    <h1>{{ appNev }}</h1>
    <p>Vue projekt sikeresen elindult.</p>
  </main>
</template>

<style scoped>
.page {
  max-width: 1100px;
  margin: 0 auto;
  padding: 32px;
  font-family: Arial, sans-serif;
}
</style>
```

5. Komponens készítése

A komponens olyan újrahasznosítható elem, mint például kártya, gomb, lista vagy űrlap.

```
// src/components/InfoCard.vue
<script setup>
defineProps({
  cim: String,
  leiras: String
})
</script>

<template>
  <div class="card">
    <h2>{{ cim }}</h2>
    <p>{{ leiras }}</p>
  </div>
</template>

<style scoped>
.card {
  border: 1px solid #ddd;
  border-radius: 14px;
  padding: 20px;
  background: white;
  box-shadow: 0 8px 24px rgba(0,0,0,.08);
}
</style>
```

Komponens használata App.vue-ban

```
<script setup>
import InfoCard from './components/InfoCard.vue'
</script>

<template>
  <InfoCard
    cim="Első komponens"
    leiras="Ez egy külön fájlban létrehozott Vue komponens."
  />
</template>
```

6. Adatkötés és változók

```
<script setup>
import { ref } from 'vue'
```

```
const nev = ref('Balázs')
const pont = ref(0)

function novel() {
  pont.value++
}
</script>

<template>
  <h1>Szia {{ nev }}!</h1>
  <p>Pontszám: {{ pont }}</p>
  <button @click="novel">+1 pont</button>
</template>
```

7. Lista megjelenítése v-for segítségével

```
<script setup>
const termekek = [
  { id: 1, nev: 'Laptop', ar: 250000 },
  { id: 2, nev: 'Monitor', ar: 80000 },
  { id: 3, nev: 'Billentyűzet', ar: 18000 }
]
</script>

<template>
  <ul>
    <li v-for="termek in termekek" :key="termek.id">
      {{ termek.nev }} - {{ termek.ar }} Ft
    </li>
  </ul>
</template>
```

8. Feltételes megjelenítés

```
<script setup>
import { ref } from 'vue'

const bejelentkezve = ref(false)
</script>

<template>
  <button @click="bejelentkezve = !bejelentkezve">
    Állapot váltása
  </button>

  <p v-if="bejelentkezve">Be vagy jelentkezve.</p>
  <p v-else>Nem vagy bejelentkezve.</p>
</template>
```

9. Űrlap v-model használatával

```
<script setup>
import { ref } from 'vue'

const nev = ref('')
const email = ref('')

function kuldes() {
  alert(`Név: ${nev.value}, Email: ${email.value}`)
}
</script>

<template>
  <form @submit.prevent="kuldes" class="form">
    <label>Név</label>
    <input v-model="nev" type="text" placeholder="Név megadása">

    <label>Email</label>
    <input v-model="email" type="email" placeholder="Email megadása">

    <button type="submit">Küldés</button>
  </form>
</template>
```

10. API hívás axios segítségével

Telepítés

```
npm install axios
```

Használat komponensben

```
<script setup>
import { ref, onMounted } from 'vue'
import axios from 'axios'

const adatok = ref([])
const toltes = ref(true)
const hiba = ref(null)

onMounted(async () => {
  try {
    const response = await axios.get('http://localhost:8000/api/termekek')
    adatok.value = response.data
  } catch (err) {
    hiba.value = 'Nem sikerült betölteni az adatokat.'
  } finally {
    toltes.value = false
  }
})
</script>

<template>
  <p v-if="toltes">Betöltés...</p>
  <p v-else-if="hiba">{{ hiba }}</p>

  <ul v-else>
    <li v-for="item in adatok" :key="item.id">
      {{ item.nev }}
    </li>
  </ul>
</template>
```

Ha CORS hiba jelenik meg, akkor a backend nem engedi a frontend címéről érkező kérést. Laravel oldalon CORS beállítást kell ellenőrizni.

11. Router alapok

Telepítés

```
npm install vue-router
```

router/index.js

```
// src/router/index.js
import { createRouter, createWebHistory } from 'vue-router'
import HomeView from '../views/HomeView.vue'
import AboutView from '../views/AboutView.vue'

const routes = [
  { path: '/', name: 'home', component: HomeView },
  { path: '/about', name: 'about', component: AboutView }
]

const router = createRouter({
  history: createWebHistory(),
  routes
})

export default router
```

main.js

```
import { createApp } from 'vue'
import App from './App.vue'
import router from './router'

createApp(App).use(router).mount('#app')
```

App.vue routerrel

```
<template>
  <nav>
    <RouterLink to="/">Főoldal</RouterLink>
    <RouterLink to="/about">Rólunk</RouterLink>
  </nav>

  <RouterView />
</template>
```

12. Bootstrap Vue projektben

```
npm install bootstrap
```

```
// src/main.js
import { createApp } from 'vue'
import App from './App.vue'
import 'bootstrap/dist/css/bootstrap.min.css'
import 'bootstrap/dist/js/bootstrap.bundle.min.js'

createApp(App).mount('#app')
```

Bootstrap kártyás layout

```
<template>
  <div class="container py-4">
    <div class="row g-3">
      <div class="col-12 col-md-6 col-lg-4">
        <div class="card shadow-sm">
          <div class="card-body">
            <h5 class="card-title">Kártya címe</h5>
            <p class="card-text">Rövid leírás.</p>
            <button class="btn btn-primary">Megnyitás</button>
          </div>
        </div>
      </div>
    </div>
  </div>
</template>
```

13. Egyszerű dizájn alapelvek

- Ne rakj mindent egy oldalra: használd a komponenseket.
- A fő tartalom legyen containerben vagy max-width beállítással.
- Kártyákhoz: border-radius, padding, árnyék, fehér háttér.
- Gomboknál legyen egyértelmű fő művelet: primary, secondary, danger.
- Táblázatnál mobilon használj görgethető wrapper divet.
- Színekből elég 2-3 fő szín: háttér, fő szín, figyelmeztető szín.

14. Gyakori hibák és javítások

Port foglalt

```
npm run dev -- --port 5174
```

Module not found

Általában rossz import útvonal vagy hiányzó npm csomag. Ellenőrizd a fájl nevét, kisbetű-nagybetű egyezést és futtasd újra:

```
npm install
```

Fehér oldal

Nyisd meg a böngésző konzolt F12-vel. Leggyakoribb ok: hibás import, hibás komponensnév vagy JavaScript hiba.

Build készítése

```
npm run build
```

Várt eredmény: létrejön egy dist mappa. Ez a kész, publikálható frontend build.

15. Környezeti változók Vue-ban

API címeket ne égesd bele minden komponensbe. Használj .env fájlt a frontend projekt gyökerében.

```
// .env
VITE_API_URL=http://127.0.0.1:8000/api

// src/services/api.js
import axios from 'axios'

export const api = axios.create({
  baseURL: import.meta.env.VITE_API_URL,
  headers: {
    'Accept': 'application/json'
  }
})
```

Használat komponensben

```
import { api } from '../services/api'

const response = await api.get('/turak')
```

16. CRUD lista komponens

```
<script setup>
import { ref, onMounted } from 'vue'
import { api } from '../services/api'

const turak = ref([])
const toltes = ref(false)
const hiba = ref(null)

async function betolt() {
  toltes.value = true
  hiba.value = null

  try {
    const response = await api.get('/turak')
    turak.value = response.data
```

```
  } catch (error) {
    hiba.value = 'Az adatok betöltése nem sikerült.'
  } finally {
    toltes.value = false
  }
}

async function torol(id) {
  if (!confirm('Biztosan törlöd?')) return
  await api.delete(`/turak/${id}`)
  await betolt()
}

onMounted(betolt)
</script>

<template>
  <section class="container py-4">
    <h1>Túrák</h1>

    <p v-if="toltes">Betöltés...</p>
    <p v-if="hiba" class="alert alert-danger">{{ hiba }}</p>

    <table class="table table-striped" v-if="!toltes">
      <thead>
        <tr>
          <th>ID</th>
          <th>Leírás</th>
          <th>Táv</th>
          <th>Aktív</th>
          <th>Művelet</th>
        </tr>
      </thead>
      <tbody>
        <tr v-for="tura in turak" :key="tura.id">
          <td>{{ tura.id }}</td>
          <td>{{ tura.leiras }}</td>
          <td>{{ tura.tav }} km</td>
          <td>{{ tura.aktiv ? 'Igen' : 'Nem' }}</td>
          <td>
            <button class="btn btn-sm btn-danger" @click="torol(tura.id)">
              Törlés
            </button>
          </td>
        </tr>
      </tbody>
    </table>
  </section>
</template>
```

17. Új rekord létrehozása űrlappal

```
<script setup>
import { ref } from 'vue'
import { api } from '../services/api'

const form = ref({
  kategoria_id: 1,
  leiras: '',
  tav: 1,
  emelkedes: 0,
  lejtes: 0,
  aktiv: true
})

const hibak = ref({})

async function mentes() {
  hibak.value = {}

  try {
    await api.post('/turak', form.value)
    alert('Sikeres mentés')
  } catch (error) {
    if (error.response?.status === 422) {
      hibak.value = error.response.data.errors
    }
  }
}
</script>
```

```
<template>
  <form @submit.prevent="mentes" class="container py-4">
    <input v-model="form.leiras" class="form-control" placeholder="Leírás">
    <small v-if="hibak.leiras" class="text-danger">{{ hibak.leiras[0] }}</small>

    <input v-model.number="form.tav" type="number" class="form-control mt-3">
    <small v-if="hibak.tav" class="text-danger">{{ hibak.tav[0] }}</small>

    <label class="form-check mt-3">
      <input v-model="form.aktiv" type="checkbox" class="form-check-input">
      Aktív
    </label>

    <button class="btn btn-success mt-3">Mentés</button>
  </form>
</template>
```

18. Keresés és szűrés frontenden

```
<script setup>
import { ref, computed } from 'vue'

const kereses = ref('')
const turak = ref([
  { id: 1, leiras: 'Erdei túra', aktiv: true },
  { id: 2, leiras: 'Városi túra', aktiv: false }
])

const szurtTurak = computed(() => {
  return turak.value.filter(t =>
    t.leiras.toLowerCase().includes(kereses.value.toLowerCase())
  )
})
</script>

<template>
  <input v-model="kereses" class="form-control" placeholder="Keresés...">

  <ul>
    <li v-for="tura in szurtTurak" :key="tura.id">
      {{ tura.leiras }}
    </li>
  </ul>
</template>
```

19. Gyors frontend ellenőrző lista

- npm run dev fut-e?
- Nincs-e piros hiba a böngésző konzolban?
- A komponensek neve és importja pontosan egyezik-e?
- API cím jó-e? Backend fut-e?
- CORS hiba esetén backend beállítást kell ellenőrizni.
- Mobilnézetben nem lóg-e ki táblázat vagy kép?